Ira A. Fulton Schools of Engineering
Arizona State University

CSE 531: Distributed and Multiprocessor Operating Systems

CSE351 Logical Clock Written Report
Casey Tom

Directions

Use this learner template to write your report for the **Logical Clock Project**. Replace the placeholder text within each section with the required information. You may add subheadings on the template to purposefully call attention to specific, organized details.

Your written report must be a single PDF with the correct naming convention: *CSE 531_Your Name_Logical Clock_Written Report.*

You must submit this report in the designated submission space in the course. Learners may **not** email or use other means to submit this project deliverable for course team review and grading. Learners should review the academic integrity and plagiarism policies prior to beginning this project.

Rubrics communicate specific criteria for evaluation. Prior to starting any graded coursework, you are expected to read through the rubric found on the project overview document, so you know how you will be assessed. You are encouraged to self-assess your work and make informed revisions before submitting. Engaging in this learning practice will support you in developing your highest quality deliverables. *Points may be deducted at the discretion of the course team for disorganized submissions that convolute the grading process.*

Problem Statement

This project involves implementing **Lamport's logical clock algorithm** by extending the distributed banking system developed in the gRPC project. The logical clock algorithm is designed to coordinate event ordering in a distributed system, ensuring that processes maintain a consistent view of event sequences despite the lack of a global clock.

gRPC Project Assumptions:

1. All customers share the same bank account.
2. Each customer interacts exclusively with a specific branch identified by a matching unique ID.
3. Transactions occur sequentially, avoiding concurrent updates to the bank account.
4. Each branch maintains a replica of the account balance that must remain consistent with all other replicas.

5. While updates are initiated independently by customers to specific branches, these updates must propagate to all branches, maintaining synchronization.

Logical Clock Project Assumptions:

1. Processes log events such as request dispatch, request reception, response dispatch, and branch propagation messages along with their logical clock values.
2. The system logs the request ID and branch ID for every message to enable reconciliation of logical clock values across processes.

Goal

The goal of this project is to extend the distributed banking system to implement Lamport's logical clock algorithm, ensuring:

- Proper ordering and synchronization of events across all processes (customers and branches).
- Accurate logging of all sent and received messages with their logical clock values.
- Maintenance of consistency in account balances across branches, validated by clock-ordered event logs.
- Comprehensive reconciliation of clock values across all processes for validation.

Major Tasks:

1. Design customer and branch processes using Python's multiprocessing module for parallelism.
2. Integrate a logical clock mechanism in all customer and branch processes.
3. Implement Lamport's clock synchronization algorithm.
4. Log every incoming and outgoing message along with the corresponding logical clock value.
5. Develop a process to initialize and manage branch servers capable of handling deposits, withdrawals, and balance queries.
6. Parse an input JSON file for initializing branch balances and processing customer requests.
7. Enable customer requests (deposit, withdrawal, and query) through inter-process communication using gRPC.
8. Define communication interfaces (e.g., Query, Deposit, Withdraw, and Propagate_Request) for interaction between customers and branches.
9. Consolidate logs from all processes into a final JSON file for validation of Lamport's clock algorithm.
10. Verify that all processes write their individual logs and that merged logs show consistent event ordering.

Setup

The setup for the project ensures a robust environment to support distributed communication, synchronization, and process management.

Development Environment:

1. **Python 3.12.3**: Used as the primary language for implementing distributed processes and gRPC communication.

2. **PyCharm Professional Edition**: An integrated development environment for writing, testing, and debugging code.
3. **Anaconda 2024.06**: Used for dependency management and creating isolated environments for project development.

Key Python Packages:

1. **Multiprocessing**: Enables concurrent customer and branch processes to simulate real-world parallelism.
2. **Concurrent.futures**: Provides high-level APIs for asynchronous task execution.
3. **gRPC (grpcio==1.64.1)**: Facilitates remote procedure calls (RPC) between distributed processes.
4. **gRPC Tools (grpcio-tools==1.64.1)**: Generates Python code from .proto files, enabling structured communication between clients and servers.
5. **Protocol Buffers (protobuf==5.27.2)**: Standardizes message serialization and ensures efficient communication.
6. **sys and json**: Used for command-line argument handling and parsing input/output data.
7. **glob**: Used for dynamically locating files based on patterns to streamline workflows and handle outputs from multiple processes.

This setup ensures seamless integration of distributed processes with accurate serialization and synchronized communication.

Implementation Processes

The implementation builds on the gRPC distributed banking system by incorporating Lamport's logical clock algorithm. Each process is designed to log and update its logical clock, ensuring consistent event ordering.

1. **Proto File Definition:**
    - The banks.proto file defines:
        - **MsgRequest**: Specifies fields for transaction ID, interface type, amount, logical clock, customer ID, and branch ID.
        - **MsgResponse**: Specifies fields for the interface type, result, amount, logical clock, customer ID, and branch ID.
    - These message types and the **Branch service** (with MsgDelivery and MsgPropagation RPCs) enable structured communication and clock synchronization.
2. **Python Code Generation Using gRPC Tools:**
    - Using grpc_tools.protoc, the .proto file is compiled into Python files:
        - banks_pb2.py for message structures.
        - banks_pb2_grpc.py for the RPC framework.
    - These files establish the foundation for gRPC-based client-server interactions.
3. **Server-Side Implementation (Branch):**
    - **Branch Class**:

        - Implements **MsgDelivery** for handling deposits, withdrawals, and balance queries, updating the branch's clock and balance.
        - Implements **MsgPropagation** for propagating updates to other branches, ensuring consistency.
    - **Clock Updates**:
        - Increments the logical clock upon sending or receiving a message.
        - Logs messages and clock values to a branch-specific JSON file.
    - **Supporting Functions**:
        - createStubs: Establishes gRPC channels for inter-branch communication.
        - ProcessRequest: Handles customer transactions and triggers propagation when necessary.
4. **Server Initialization (Server.py):**
    - Reads branch configurations from gprc_input.json and initializes branch processes.
    - Each branch process executes branch_serve, where gRPC channels and stubs are set up.
5. **Client-Side Implementation (Customer):**
    - **Customer Class**:
        - Manages gRPC stubs for communication with a branch.
        - Sends requests (deposit, withdrawal, query) and updates the logical clock before sending and after receiving messages.
    - Logs each request and response along with clock values in a customer-specific JSON file.
6. **Client Initialization (Client.py):**
    - Parses customer configurations from gprc_input.json and initializes customer processes.
    - Consolidates customer and branch logs into a final output file (output.json) for clock consistency validation.

Results

The results demonstrate the successful implementation of Lamport's logical clock algorithm. The output.json file contains logs from all processes, validating the correctness of logical clock updates.

Sample Outputs:

1. **Customer Logs (Part 1):**

```
json
Copy code
{
  "id": 1,
  "type": "customer",
  "events": [
    {
```

```
            "customer-request-id": 1,
            "logical_clock": 1,
            "interface": "deposit",
            "comment": "event_sent from customer 1"
        },
        {
            "customer-request-id": 2,
            "logical_clock": 5,
            "interface": "withdraw",
            "comment": "event_sent from customer 1"
        }
    ]
}
```

2. **Branch Logs (Part 2):**

json
Copy code

```
{
    "id": 1,
    "type": "branch",
    "events": [
        {
            "customer-request-id": 1,
            "logical_clock": 2,
            "interface": "deposit",
            "comment": "event_recv from customer 1"
        },
        {
            "customer-request-id": 1,
            "logical_clock": 3,
            "interface": "propogate_deposit",
            "comment": "event_sent to branch 2"
        }
    ]
}
```

3. **Overall Logs (Part 3):**

json
Copy code

```
{
    "id": 3,
    "type": "branch",
    "events": [
        {
```

```
            "customer-request-id": 1,
            "logical_clock": 5,
            "interface": "propogate_deposit",
            "comment": "event_recv from branch 1"
        },
        {
            "customer-request-id": 3,
            "logical_clock": 12,
            "interface": "propogate_deposit",
            "comment": "event_recv from branch 2"
        }
    ]
}
```


Detailed Analysis:

1. Logical clocks increment correctly for each event, following Lamport's algorithm.
2. Clocks synchronize across processes by updating to max(local clock, received clock) + 1.
3. The logs validate that all sent and received messages maintain clock consistency.

Justification: The results show that Lamport's logical clock algorithm is correctly implemented, ensuring accurate event ordering and synchronization in the distributed system. Every customer and branch log confirms proper clock updates and alignment between processes.